CS6P008 - Deep Learning Lab

Lab 3, Date: Feb 02, 2026

Topic(s): Single layer perceptron

Timing: 10:00 AM to 01:00 PM — Max Points: 10

Instructions

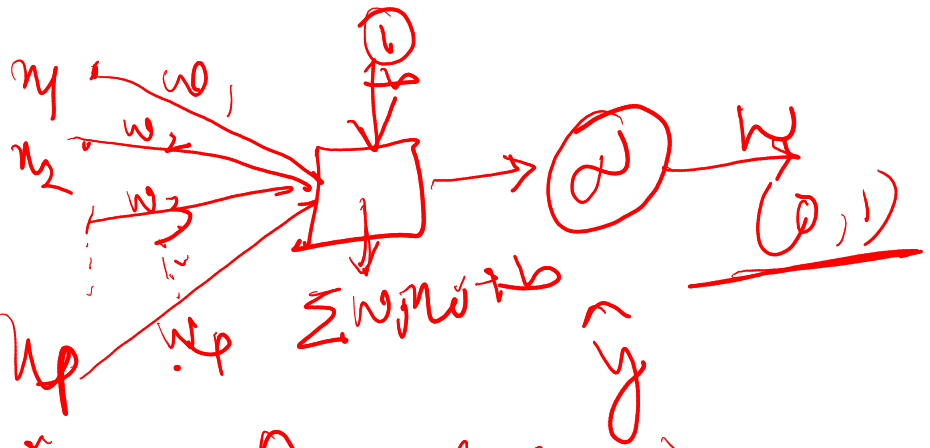
1. Provide commented, indented code. Variables should have meaningful names.
2. Submit one .ipynb file containing all answers.
   The name should be `[student name]_lab[number].ipynb`
3. Read the questions carefully before answering. If a question asks to follow a particular approach or to use a specific data structure, then it must be followed.
4. Write questions in separate text blocks in Jupyter/Colab Notebook before the code blocks containing answers.
5. All plots should have appropriate axis labels, titles, and legends.

Do not use the solutions provided by third-party libraries such as scikit-learn. You can use `numpy`, `pandas`, `matplotlib`.

1. Implement the **single-layer Perceptron algorithm from scratch without using any Library function** and train the model on the **Pima Indians Diabetes Database** (Dataset Link). Take 80% data for training and the rest for testing. (5)
   Tasks:
   - Use the **Sigmoid activation function** for the Perceptron model: $\hat{y} = \sigma(\mathbf{w}^T\mathbf{x}+b)$.
   - Train the model using **Stochastic Gradient Descent (SGD)** as the optimizer. Write the learning rule?
   - Experiment with at least three different learning rates until convergence. Use a learning rate scheduler such that the learning rate is reduced by a factor of 10 if the training loss does not decrease for five consecutive epochs.
   - Implement Accuracy, Precision, Recall, and F1-Score from scratch. Compare the Classification report (Accuracy, Precision, Recall and F1 Score) to find out the best performing model.

Table 1: Performance of Perceptron Model for Different Learning Rates and Epochs

η	**Epochs**	**Accuracy**	**F1-score**	**Precision**	**Recall**	**Overfitted (Y/N)**



$$\hat{y} = \sigma(w^T x + b)$$

$$\sigma(\theta^T x)$$

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \end{bmatrix} \qquad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix} \qquad \theta = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \\ b \end{bmatrix}$$

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta l(\theta)}$$

$(p+1)$ $\quad$ $p+1$ $\quad$ Scalar $\quad$ $p+1$

$$\begin{bmatrix} \frac{\partial l(\theta)}{\partial w_1} \\ \frac{\partial l(\theta)}{\partial w_2} \\ \frac{\partial l(\theta)}{\partial w_3} \\ \vdots \end{bmatrix}$$

$$l(\theta) = -\left(y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right)$$

$$\nabla_\theta l(\theta) = \frac{\partial l(\theta)}{\partial \theta}$$

$$l(\theta) \text{ of } (\hat{y}_i)$$

$$\hat{y}_i = \sigma(z_i)$$

$$z_i = w^T x_i = \theta^T x_i$$

$$\frac{\partial l(\theta)}{\partial \theta} = \frac{\partial l(\theta)}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \theta}$$

$$= \left[\frac{y_i}{\hat{y}_i} - \frac{(1-y_i)}{(1-\hat{y}_i)} \right] \hat{y}_i (1-\hat{y}_i) x_i$$

$$= \left[(1-y_i)\hat{y}_i - y_i (1-\hat{y}_i) \right] x_i$$

$$= (\hat{y}_i - y_i) x_i$$

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta l(\theta)$$

$$= \theta_t - \eta (\hat{y}_i - y_i) x_i$$

$p+1$ $\quad$ $p+1$ $\quad$ 1 $\quad$ $(0,1)$ $\quad$ $1/0$ $\quad$ $p+1$

$$\begin{bmatrix} x_{i1} \\ x_{i2} \\ x_{i3} \\ \vdots \\ x_{ip} \\ 1 \end{bmatrix}$$

$l(\theta)$

$$\begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ b \end{bmatrix}_{t+1} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ b \end{bmatrix}_{t} - \eta (\hat{y}_i - y_i) \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \\ 1 \end{bmatrix}$$

$t=0$, w_i, b initialize

$w_i \in U[-1,1]$, $w_i \neq 0$

$b \in U[-1,1]$, $b \neq 0$

$\eta = 0.1, 0.01, 0.001$

2. Plot an epoch vs loss curve for each of the training and validation splits of your dataset. Justify your answer based on the observed training results. (2)

3. Similar to the previous experiment, use any standard Python library to replicate the best-performing Perceptron model identified earlier. Train the model using the same training dataset and the previously obtained best hyperparameters. (3)

Table 2: Comparison of Perceptron Models on Test Dataset

Model	**Accuracy**	**Precision**	**Recall**	**F1-score**
From-scratch				
Library-based				